

SkillOpt for AgentIA

A bounded text-space optimization layer for procedural reliability

AgentIA Engineering

September 25, 2026

In one paragraph. AgentIA generates Java 21 / Spring Boot 3 microservice scaffolding and validates it with `mvn test -o`. Its recurring failures are procedural: the model *knows* a rule (use `jakarta.*`, not `javax.*`; controllers must not inject repositories directly) but forgets to apply it in a given session. We propose adding a *skill layer* — a small set of triggerable procedural documents prepended to the generation prompt — and, once its value is demonstrated, an optimizer that improves those documents under a held-out validation gate. The optimizer is SkillOpt (Microsoft Research, 2026). The plan is staged: *inject and measure* first (1–2 weeks); *optimize* only if the measurement shows signal (4 weeks).

1 The problem

AgentIA’s generation pipeline ends in VERIFIED or BLOCKED. A BLOCKED session has exhausted the bounded repair loop (max 3 attempts) and requires human intervention. Two distinct failure classes produce this outcome:

Capability gap. The model does not know the rule. A better reminder will not help, because there is nothing to remind.

Application gap. The model knows the rule but fails to apply it in a specific session. A persistent, triggered reminder can help.

The SkillOpt layer targets the *second* class. This distinction matters for scope: the mechanism improves reliability within what AgentIA already covers, not coverage of what it does not (Spring Security configuration, reactive programming, Saga/CQRS, the Azure stack). Coverage expansion is a separate investment.

2 What a skill is

A skill is *not* a cheat sheet (static facts). It is a *procedural policy* with three parts:

1. **Trigger** (`when_to_apply`) — the situation in which the skill becomes relevant.
2. **Procedure** (`rules`) — an ordered set of actions to take once the trigger fires.
3. **Granularity** — task-level (applies to a whole task) or event-driven (fires on a specific execution signal).

Representative skill (adapted from CODESKILL’s Java/Maven examples):

Title. Handle missing Maven test-jar dependencies.

When to apply. A Maven build fails with a dependency-resolution error for a test-jar that has not yet been produced.

Rules. (1) Inspect the failing module's POM to confirm dependency type and scope. (2) Check the producing module's POM for an existing plugin execution. (3) If the test jar is only generated when tests run, run tests or invoke the test-jar goal explicitly. (4) Do not retry with tests skipped — skipping prevents the artifact. (5) Run a full install so the artifact lands in the local repository. (6) Verify presence before retrying the original command.

At deployment, skills are *prepended* to the generation prompt. The target model remains frozen. No inference-time overhead beyond the prompt tokens.

3 What SkillOpt is

SkillOpt treats the skill document as a **trainable parameter** and builds a training loop around it — in text space, with the same discipline that makes weight-space optimization reproducible.

3.1 The four controls

Bounded edit budget (learning rate). Each step applies at most L_t atomic edits (default 4, cosine decay, floor 2). Prevents unbounded rewriting; makes each step small enough that its effect is measurable.

Held-out validation gate (trust region). Every candidate document is scored on a *selection split* — sessions never used to motivate the edits. Accepted only if the score *strictly* improves. Ties rejected. This is what makes the loop auditable and non-degrading.

Rejected-edit buffer (negative feedback). Rejected edits and their score drops are stored and fed back to the next reflection call, so the optimizer avoids repeating failed directions.

Epoch-wise slow/meta update (momentum). At each epoch boundary, the same 20 tasks are re-run under the previous and current documents; the optimizer writes a *longitudinal guidance block* into a protected section that step-level edits cannot touch. Catches drift that adjacent-step reflection cannot see.

3.2 The loop

Inner loop (per step).

rollout on training set → split into success/failure minibatches → optimizer proposes edits → hierarchical merge, dedupe, rank → clip to top L_t → apply → candidate → GATE: accept iff selection score strictly improves, else reject and log.

Outer loop (per epoch, 4 total).

sample 20 tasks → re-run under previous and current skill → categorize regressions / persistent failures / improvements / stable successes → optimizer writes longitudinal guidance → write to protected section → same gate.

3.3 What it produces

A single `best_skill.md`, roughly 300–2,000 tokens, fully inspectable. No inference-time model calls; the document is prepended to the prompt and nothing else changes at deployment.

4 The evidence

The paper’s evaluation is unusually broad: six benchmarks, seven target models, three execution harnesses. Headline results:

- Best or tied-best on **52 of 52** evaluated (model, benchmark, harness) cells.
- Average gain on GPT-5.5 direct chat: +23.5 points over no-skill; +24.8 under Codex; +19.1 under Claude Code.
- Beats an *oracle* baseline that picks the best of six competing methods per cell by +5.4 points.
- **Edit economy:** 1–4 accepted edits (median 2.5) produce +29 to +39 points on procedural benchmarks. OfficeQA’s +39.0 arises from a single accepted edit.
- **Cross-harness transfer:** a skill trained under Codex transfers to Claude Code with +59.7 over baseline — exceeding the in-domain result.

4.1 The ablation that matters most

Removing both the meta-skill and the slow update drops SpreadsheetBench from 77.5 to 55.0 — a 22.5 point degradation, the largest in the paper. The epoch-level longitudinal view is load-bearing, not decorative.

5 Honest boundaries

- **No code benchmark.** None of the six benchmarks is code. The mechanism is domain-agnostic (it operates on text plus a verifier), but the numbers do not transfer. AgentIA’s pilot would be the first application to Java generation.
- **Requires an automated verifier.** The paper’s Limitations section is explicit. `mvn test -o` is exactly this.
- **One document, not a bank.** SkillOpt optimizes a single skill artifact. Bank-level curation (which skills to keep or retire) is a different problem (SkillBrew, Ratchet) deferred to later specs.
- **No weight changes.** This is a feature: no training infrastructure, no data pipeline, no fine-tuning.

6 The staged plan

6.1 Spec 1 — inject and measure (1–2 weeks)

1. **Task 0.** Query `studio.db` for the historical pass/fail distribution and repair-parser error classification. Triage the five candidate domains: skill-layer appropriate, deterministic-fixer appropriate, or not observed.
2. Hand-write seed skills for surviving domains.
3. Add `skills` table; add `skill_injection_node` before `validator_node`; log every injection (`session_id`, `skill_ids`, `versions`, `injection_point`).
4. Re-inject on every repair attempt.
5. Run a four-condition A/B pilot (minimum 15 sessions per condition): no-skills, skills, sham injection (isolates “longer prompt” from “this content”), skills + deterministic fixer.

6. **Primary metric:** `mvn test -o` pass rate. **Secondary:** reviewer tag (as-is / minor edits / rewrite).

6.2 Spec 2 — optimize (4 weeks, only if Spec 1 shows signal)

Build the SkillOpt loop: rollout collector, reflector (using the paper’s published prompts), merger, ranker, applier, gate, rejected-edit buffer, slow/meta update. Optimizer model is a *different* provider than the target, routed through the existing LLMFactory.

7 The pilot in detail

Condition	Prompt addition	Purpose
<code>no_skills</code>	none	Baseline
<code>skills</code>	active skill documents	Treatment
<code>sham</code>	~1.5K tokens of irrelevant text	Prompt-length control
<code>skills_plus_fixer</code>	skills + deterministic post-processor	Fixer interaction

Causal design. Random assignment within the same agent version and time window, not skill-on-now vs. skill-off-historically. **Skill versioning.** Freeze skill content for the full pilot window; version-tag every session.

8 Decision points

1. **After Task 0.** Which domains survive triage? Which are better handled by deterministic fixers?
2. **After Spec 1.** Does injection move the primary metric? If no, stop. If yes, build Spec 2.
3. **Before Spec 2.** Spec 2 versus coverage expansion (Spring Security config generation) — evaluated on equal footing against the original economic framing, not by momentum.

The arithmetic worth stating explicitly. Reliability improvements on a 30–40% covered slice cannot mathematically reach a target defined against 100% of the requisition while the remaining 60–70% is uncovered. The reliability work is necessary infrastructure for any coverage expansion, but it is not a substitute for it. The resourcing decision after Spec 1 should reflect that.

9 References

- Li et al. *SkillOpt: A controllable text-space optimizer for agent skills*. arXiv:2605.23904, 2026.
- Li et al. *CODESKILL: Learning self-evolving skills for coding agents*. arXiv:2605.25430, 2026.
- Hu et al. *SkillBrew: Multi-objective curation of skill banks for LLM agents*. arXiv:2605.29440, 2026.
- Eliseeva et al. *EnvBench: A benchmark for automated environment setup*. arXiv:2503.14443, 2025.